

React-Native Flexbox

Desenvolvimento para Dispositivos Móveis

Prof. Me. Jackson Meires
jackson.meires@ifsc.edu.br

Slides adaptados do Prof. Me. Fernando Albiero
fernando.albiero@ifsc.edu.br

Flexbox

Um componente pode especificar o layout dos seus filhos utilizando o algoritmo de flexbox

O flexbox foi projetado para fornecer um layout consistente em diferentes tamanhos de tela

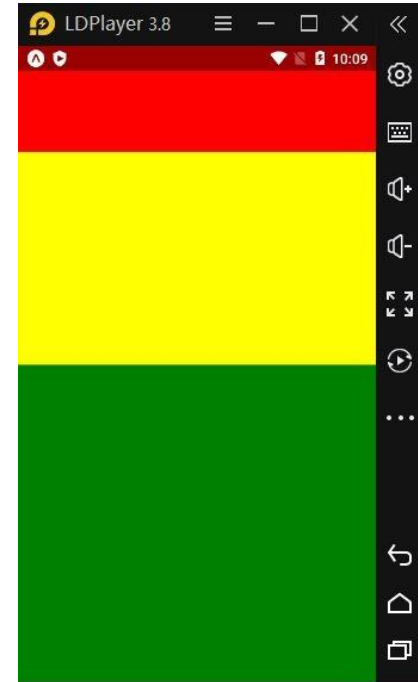
Normalmente usa-se uma combinação das propriedades `flexdirection`, `alignItems` e `justifyContent` para alcançar o layout desejado

Flex

Define o espaço que cada componente irá ocupar tela

Considere o exemplo ao lado, onde tem-se 3 componentes: 1 vermelho, 1 amarelo e 1 verde

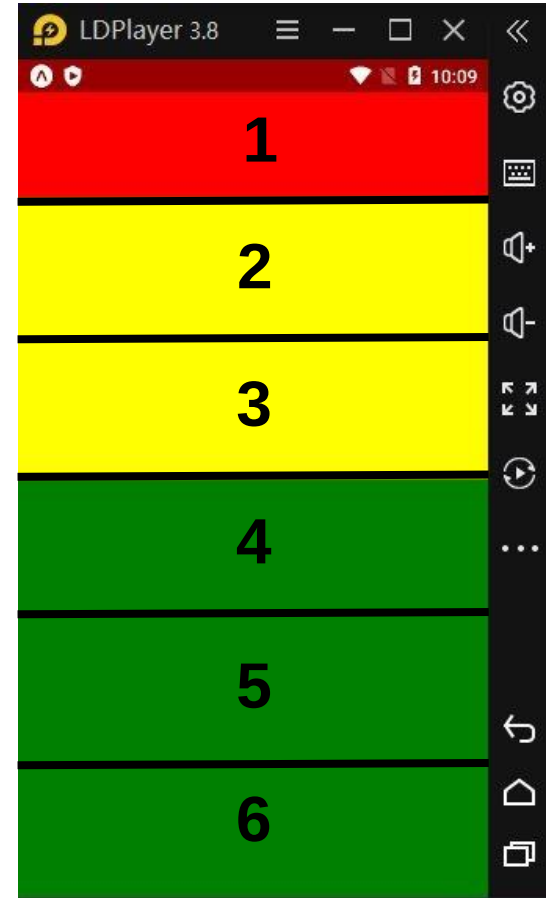
Cada um possui um valor diferente para a propriedade flex



Vermelho possui flex: 1, amarelo flex: 2 e verde flex: 3

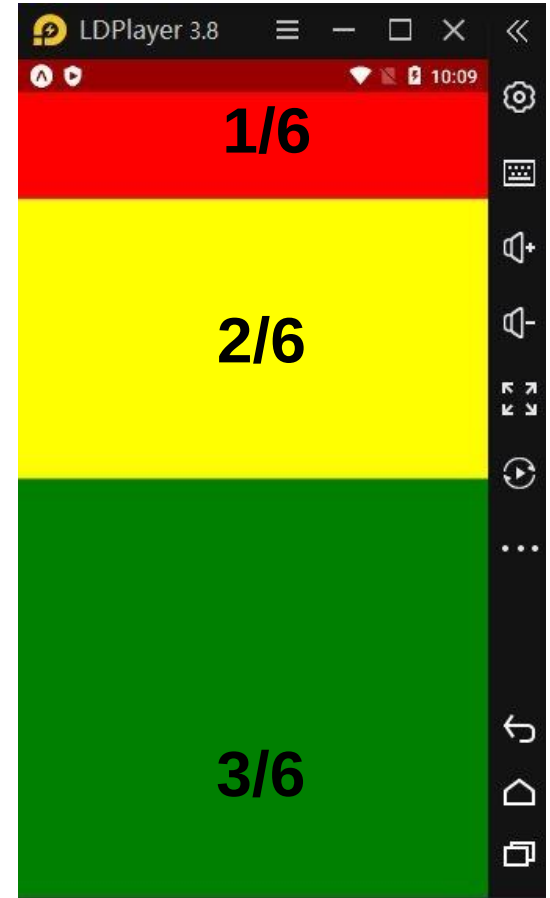
O que o algoritmo faz é somar o flex de todos os componentes mostrados na tela e dividir a tela em espaços de mesmo tamanho, de acordo com a soma obtida, no caso: $1 + 2 + 3 = 6$

Ou seja, a tela está dividida em 6 pedaços de tamanho igual.



Quando dizemos que vermelho possui um flex: 1, o que estamos falando na verdade é que ele ocupará $1/6$ da tela

Logo, o componente amarelo ocupará $2/6$ (flex 2) e o verde $3/6$ (flex 3).



```
import React from 'react';
import { StyleSheet, View } from 'react-native';

export default class App extends React.Component {
  constructor(props){
    super(props);
  }

  render(){
    return (
      <View style={styles.container}>
        <View style={styles.vermelho}></View>
        <View style={styles.amarelo}></View>
        <View style={styles.verde}></View>
      </View>
    );
  }
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1
  },
  vermelho: {
    flex: 1,
    backgroundColor: 'red'
  },
  amarelo: {
    flex: 2,
    backgroundColor: 'yellow'
  },
  verde: {
    flex: 3,
    backgroundColor: 'green'
  }
});
```

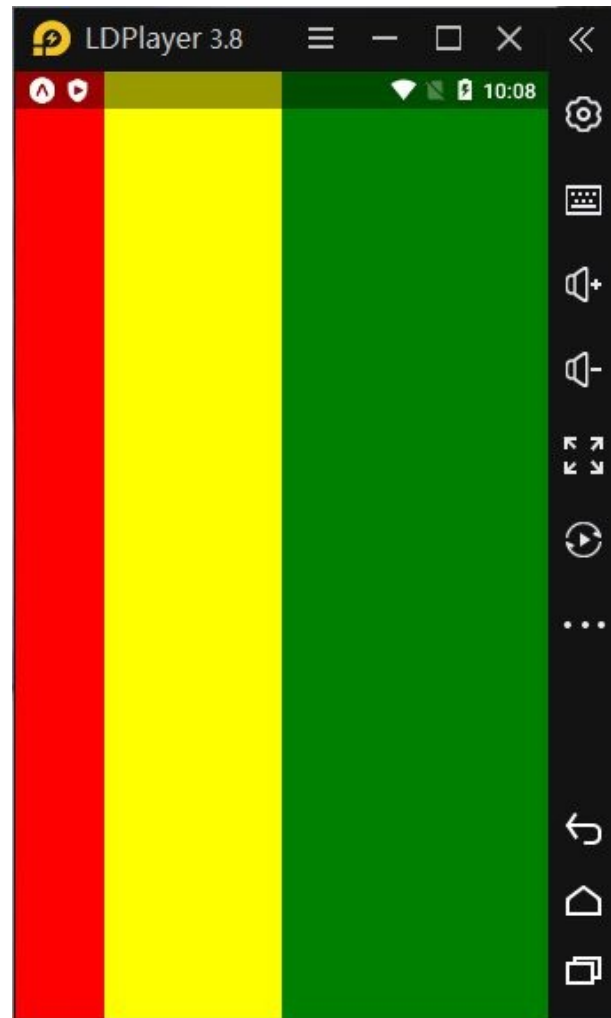
flexDirection

Controla **direção em que os componentes mostrados na tela**

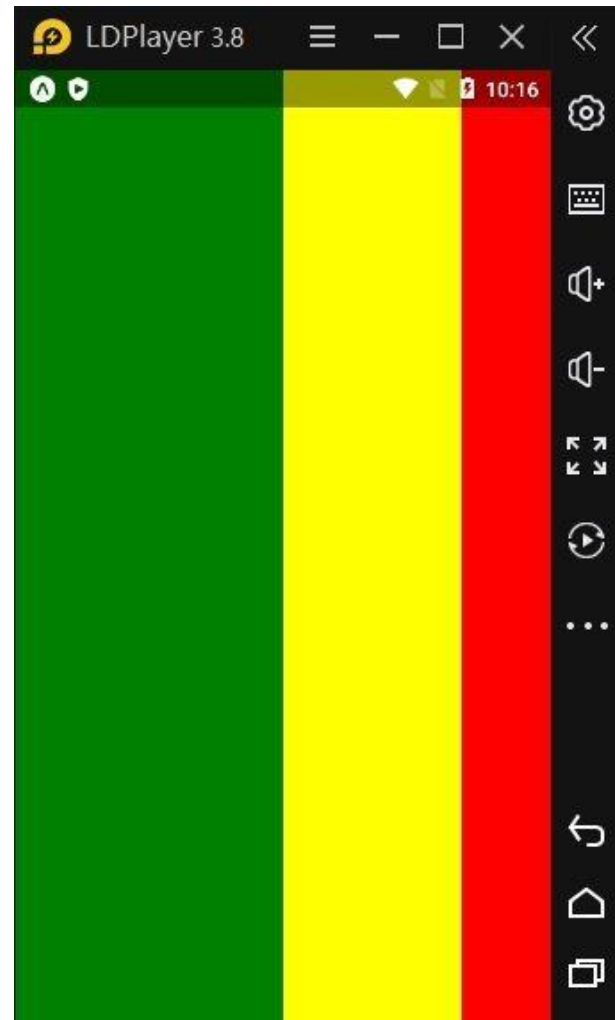
Pode assumir quatro valores: row, column, row-reverse ou column-reverse.

Por padrão o valor é column

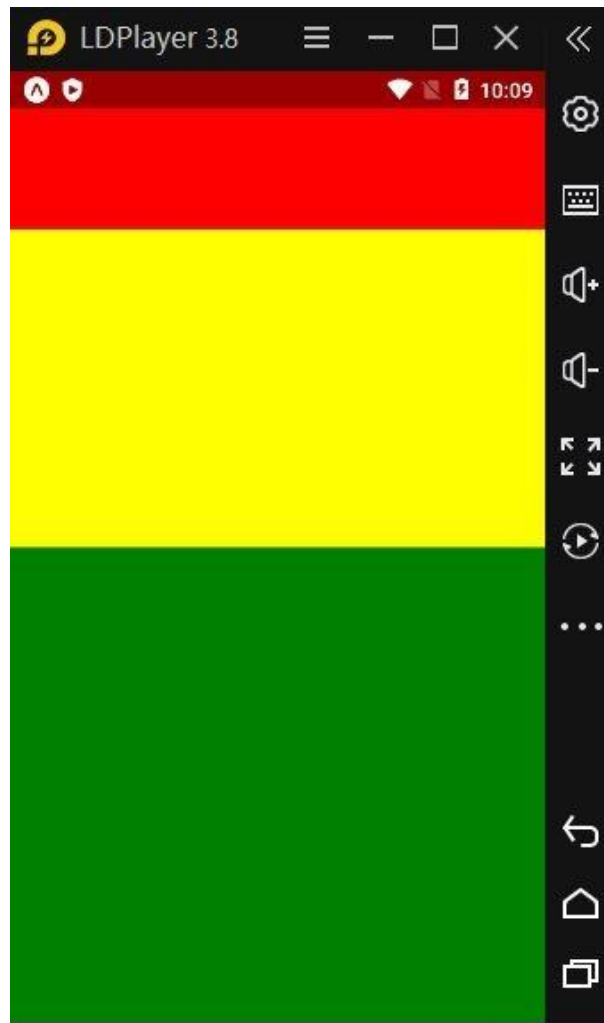
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "row"
  },
  vermelho: {
    flex: 1,
    backgroundColor: 'red'
  },
  amarelo: {
    flex: 2,
    backgroundColor: 'yellow'
  },
  verde: {
    flex: 3,
    backgroundColor: 'green'
  }
});
```



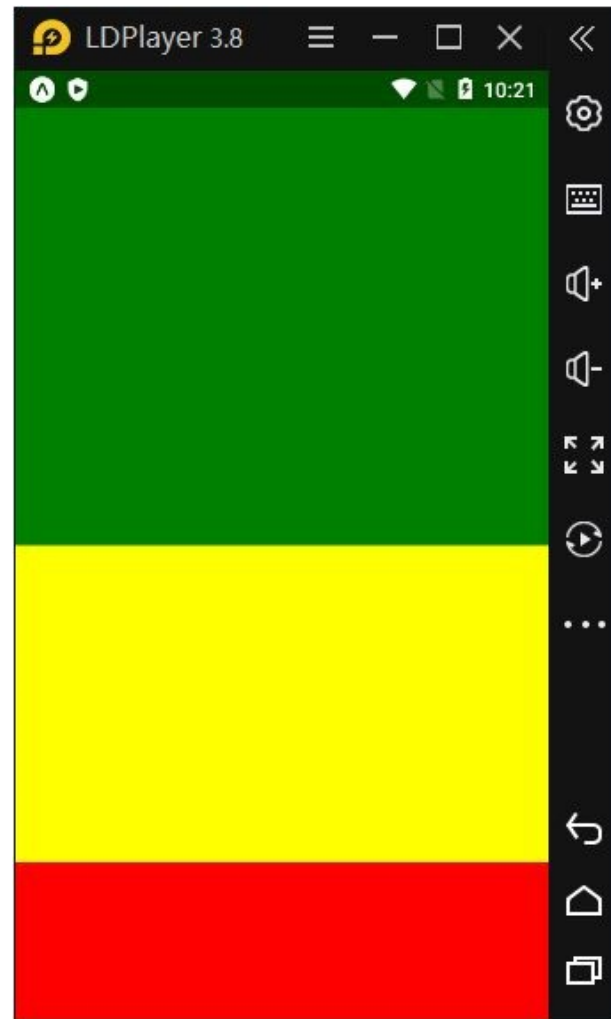
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "row-reverse"
  },
  vermelho: {
    flex: 1,
    backgroundColor: 'red'
  },
  amarelo: {
    flex: 2,
    backgroundColor: 'yellow'
  },
  verde: {
    flex: 3,
    backgroundColor: 'green'
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column"
  },
  vermelho: {
    flex: 1,
    backgroundColor: 'red'
  },
  amarelo: {
    flex: 2,
    backgroundColor: 'yellow'
  },
  verde: {
    flex: 3,
    backgroundColor: 'green'
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column-reverse"
  },
  vermelho: {
    flex: 1,
    backgroundColor: 'red'
  },
  amarelo: {
    flex: 2,
    backgroundColor: 'yellow'
  },
  verde: {
    flex: 3,
    backgroundColor: 'green'
  }
});
```



justifyContent

Controla **alinhamento em que os componentes serão mostrados na tela. Leva em consideração o flexDirection (alinhamento vertical ou horizontal)**

Pode assumir seis valores: flex-start, flex-end, center, space-between, space-around, space-evenly.

Por padrão o valor é flex-start

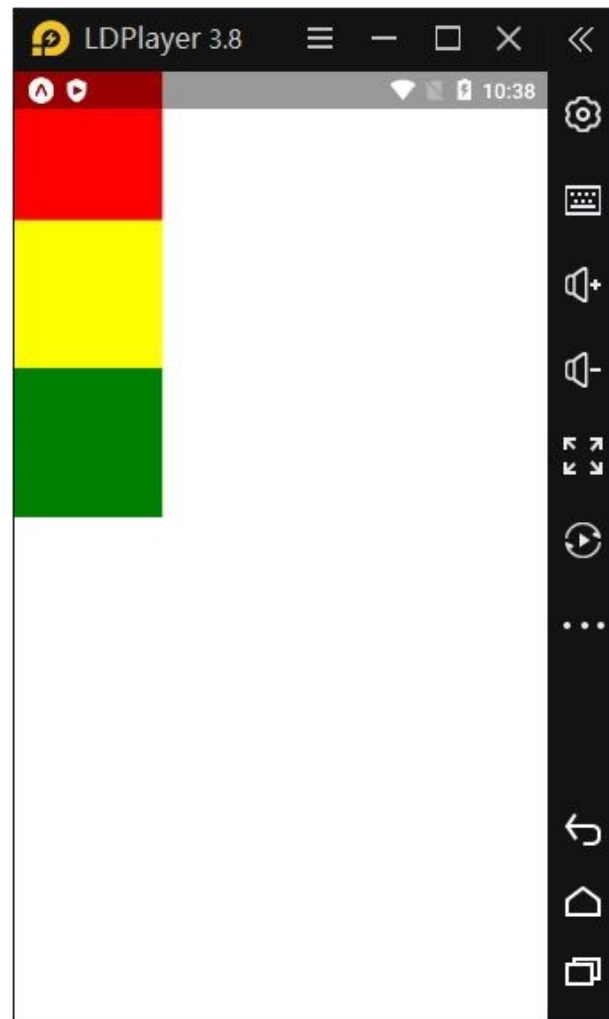
width e height

Usados para **comprimento e altura do componente,**
respectivamente

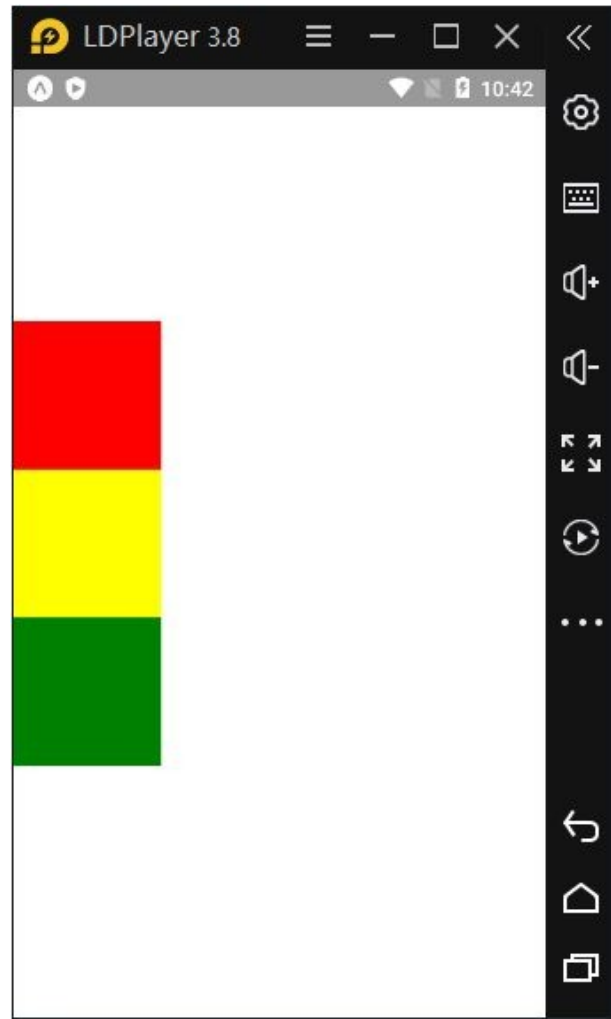
Podem assumir os valores: pixel, porcentagem ou auto

O valor padrão é auto, onde o tamanho do componente é calculado automaticamente baseado no seu conteúdo.

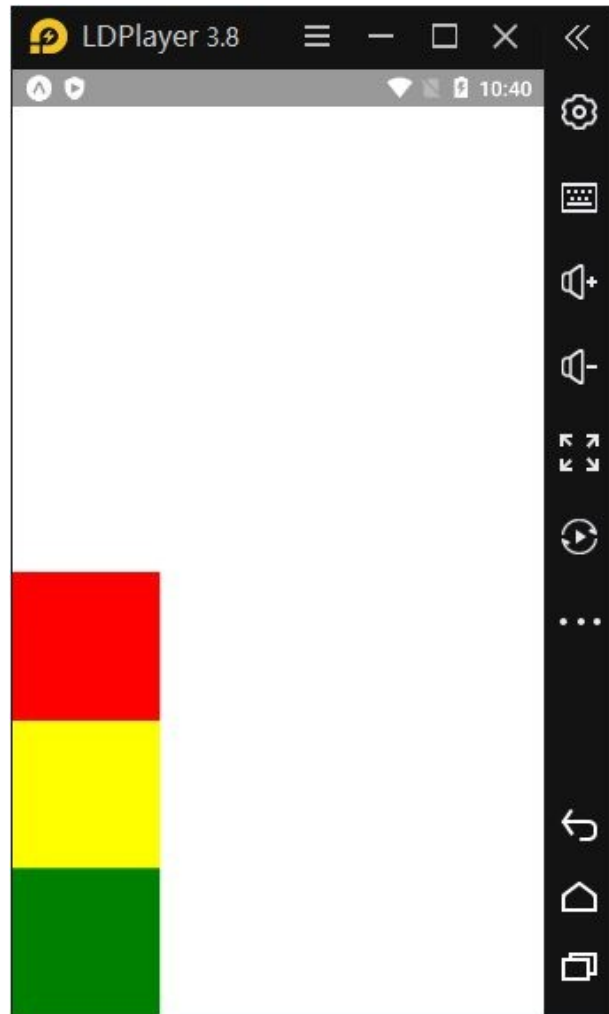
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    justifyContent: "flex-start"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



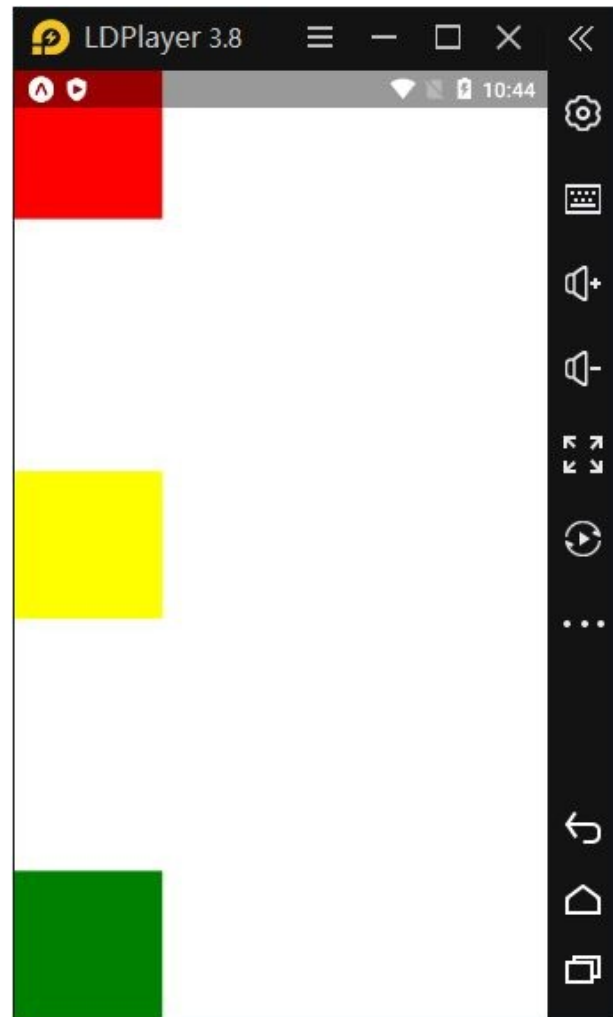
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    justifyContent: "center"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



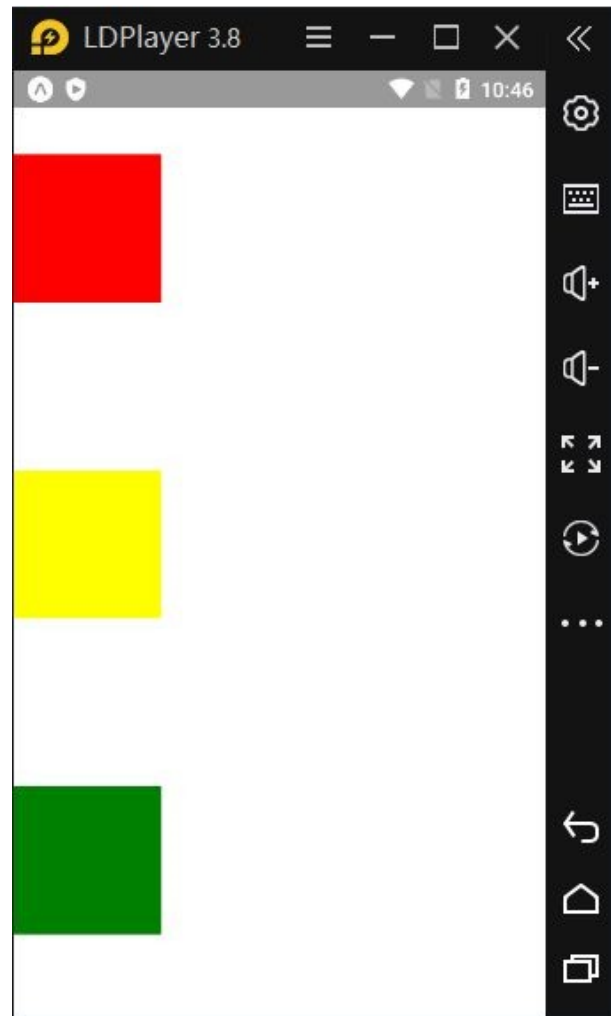
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    justifyContent: "flex-end"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    justifyContent: "space-between"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    justifyContent: "space-around"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



alignItems

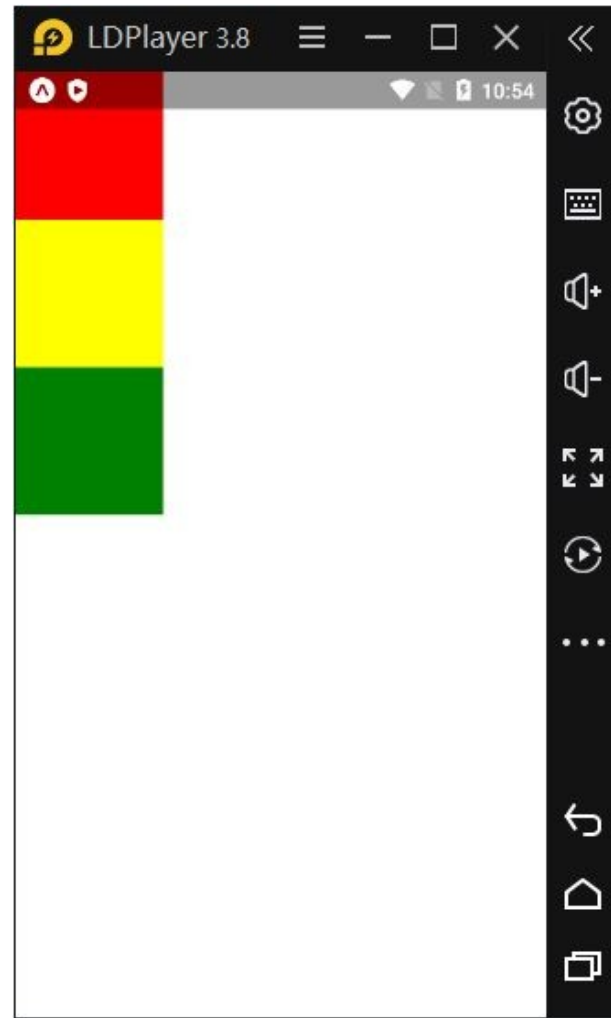
Controla o **alinhamento horizontal dos componentes.**

Semelhante ao justifyContent, porém não leva em consideração o **flexDirection**, o alinhamento será sempre com base no eixo x

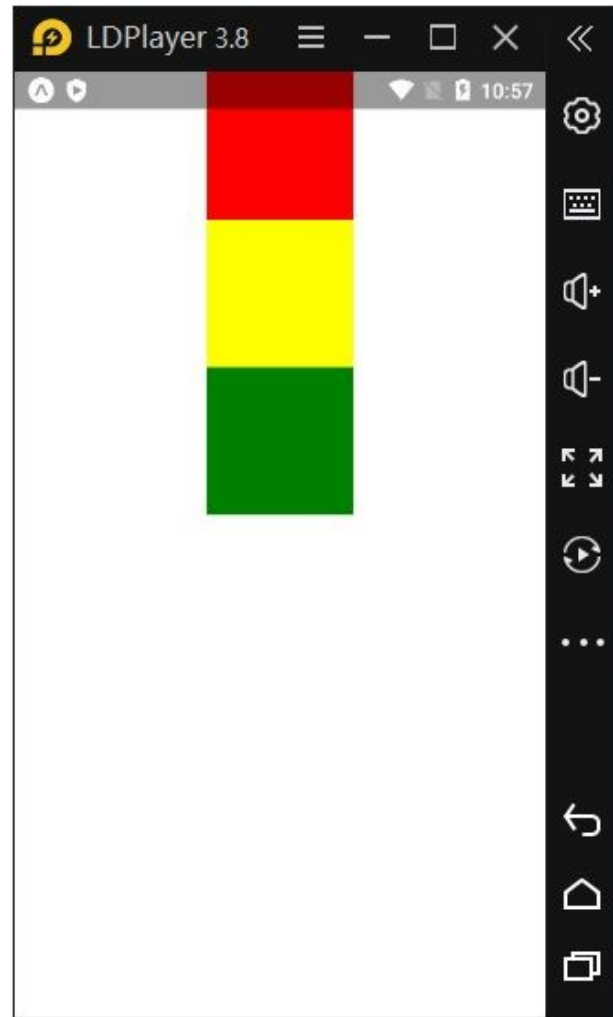
Pode assumir cinco valores: stretch (default), flex-start, flex-end, center, baseline.

Por padrão o valor é stretch

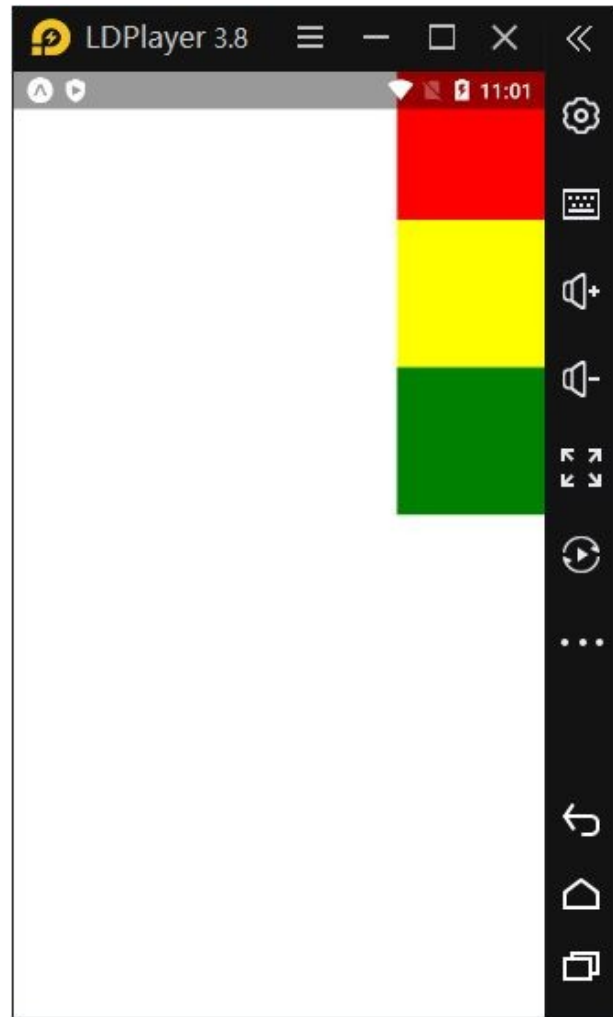
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    alignItems: "flex-start"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



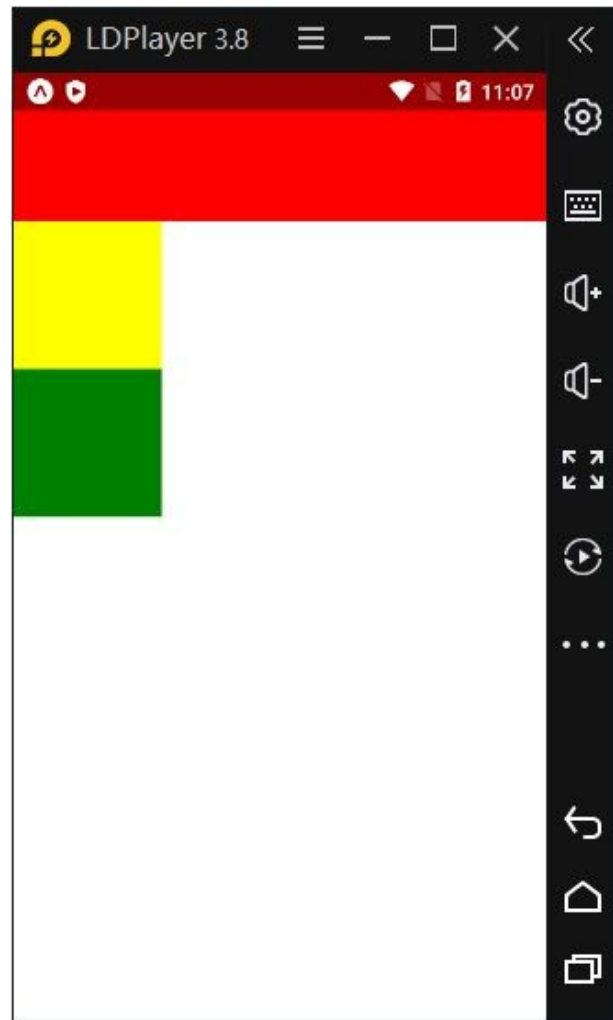
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    alignItems: "center"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    alignItems: "flex-end"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    alignItems: "stretch"
  },
  vermelho: {
    backgroundColor: 'red',
    height: 100
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 100
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 100
  }
});
```



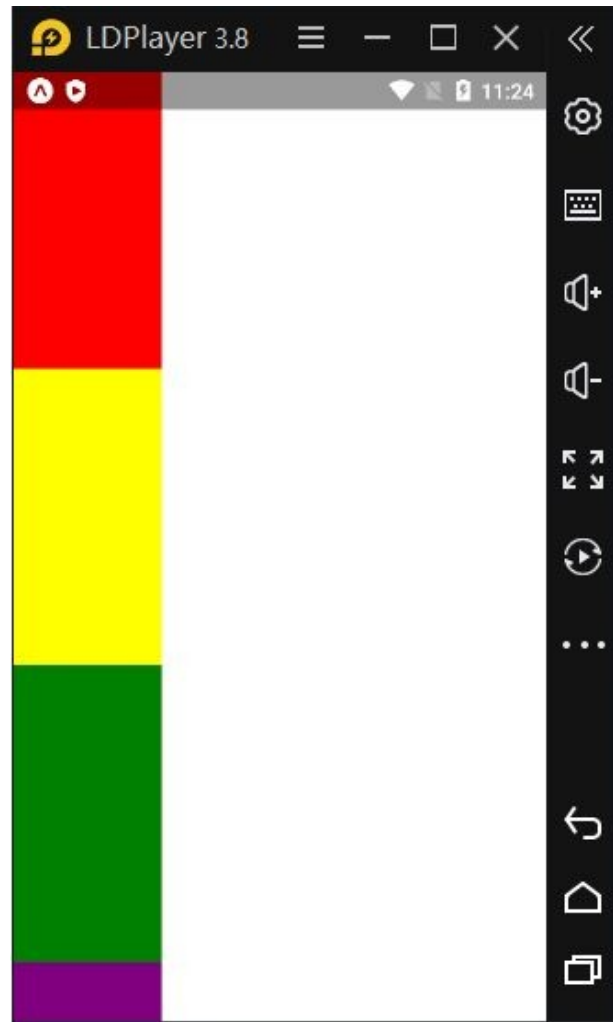
flexWrap

Propriedade que define se a quebra de linha deve ocorrer quando os componentes extrapolam o tamanho da tela.

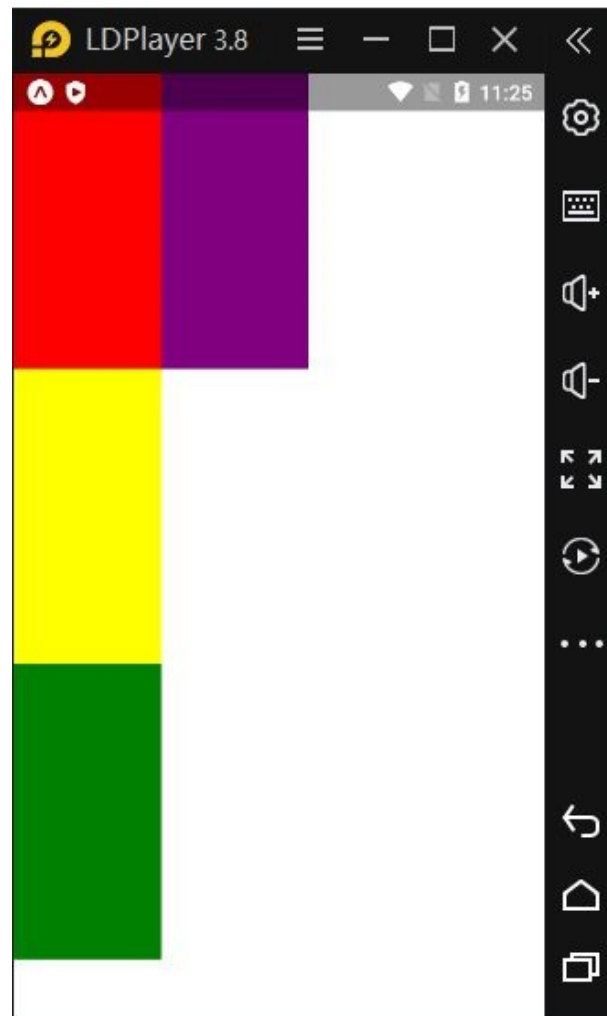
Pode assumir dois valores: wrap e nowrap

Por padrão o valor é nowrap

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "nowrap"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 200
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 200
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 200
  },
  roxo: {
    backgroundColor: 'purple',
    width: 100,
    height: 200
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "wrap"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 200
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 200
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 200
  },
  roxo: {
    backgroundColor: 'purple',
    width: 100,
    height: 200
  }
});
```



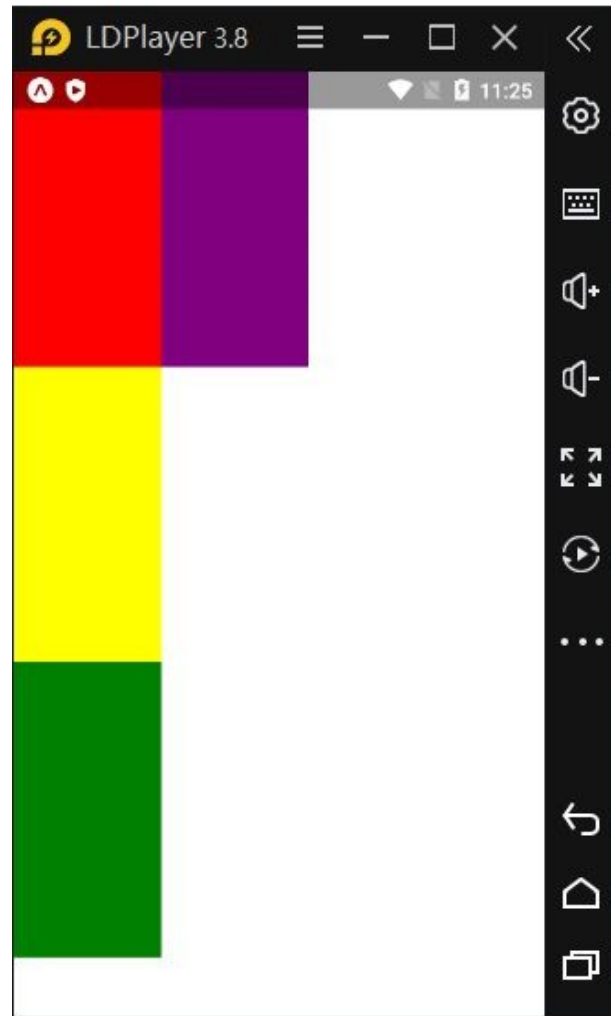
alignContent

Quando **propriedade flexWrap está definida,**
a **controlar alinhamento de cada linha do conteúdo**

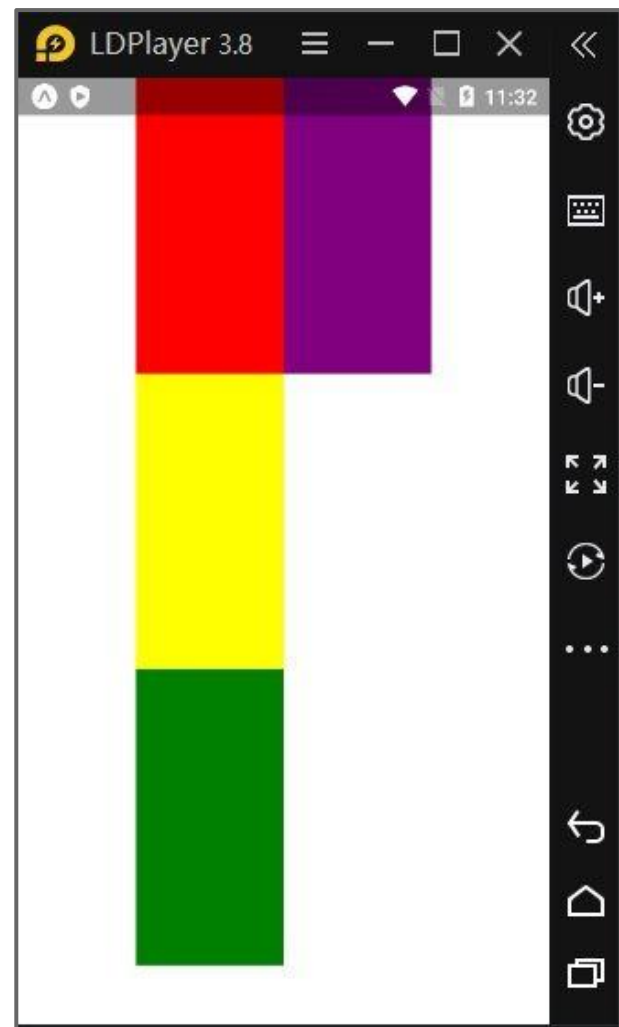
Pode assumir cinco valores: stretch (default), flex-start, flex-end, center, space-between, space-around

Por padrão o valor é flex-start

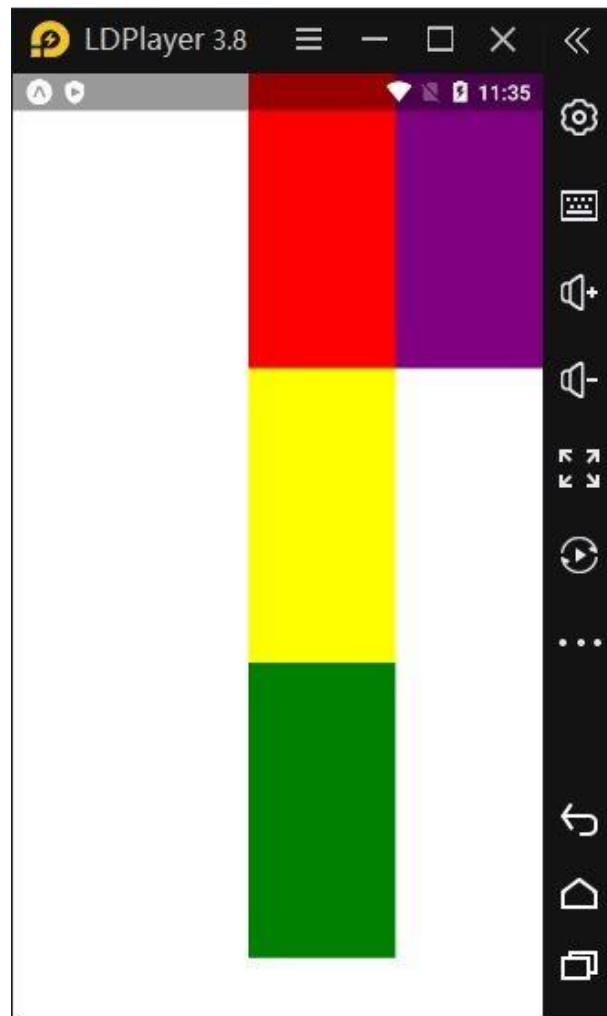
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "wrap",
    alignContent: "flex-start"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 200
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 200
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 200
  },
  roxo: {
    backgroundColor: 'purple',
    width: 100,
    height: 200
  }
});
```



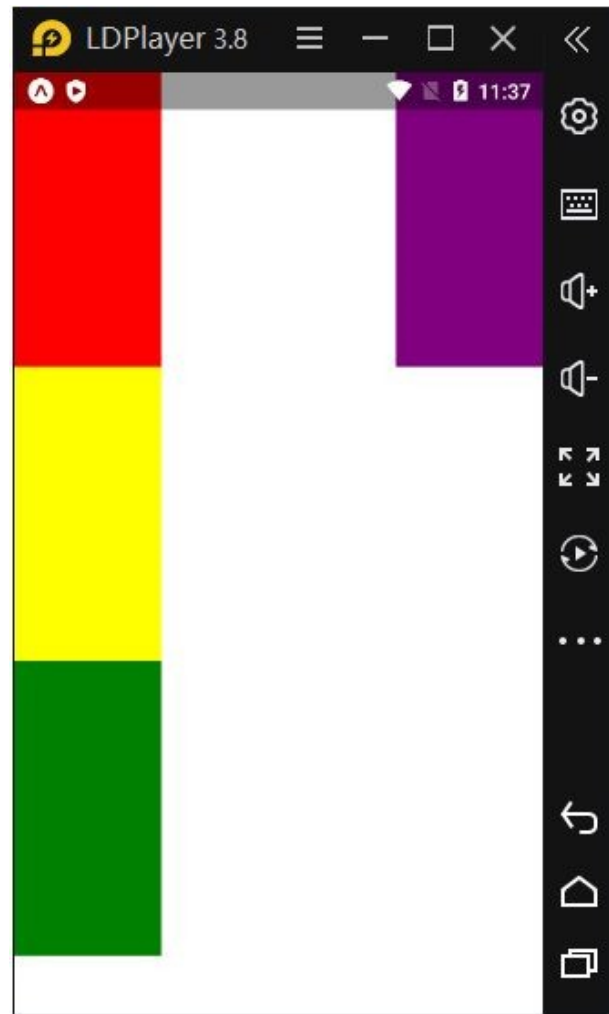
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "wrap",
    alignContent: "center"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 200
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 200
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 200
  },
  roxo: {
    backgroundColor: 'purple',
    width: 100,
    height: 200
  }
});
```



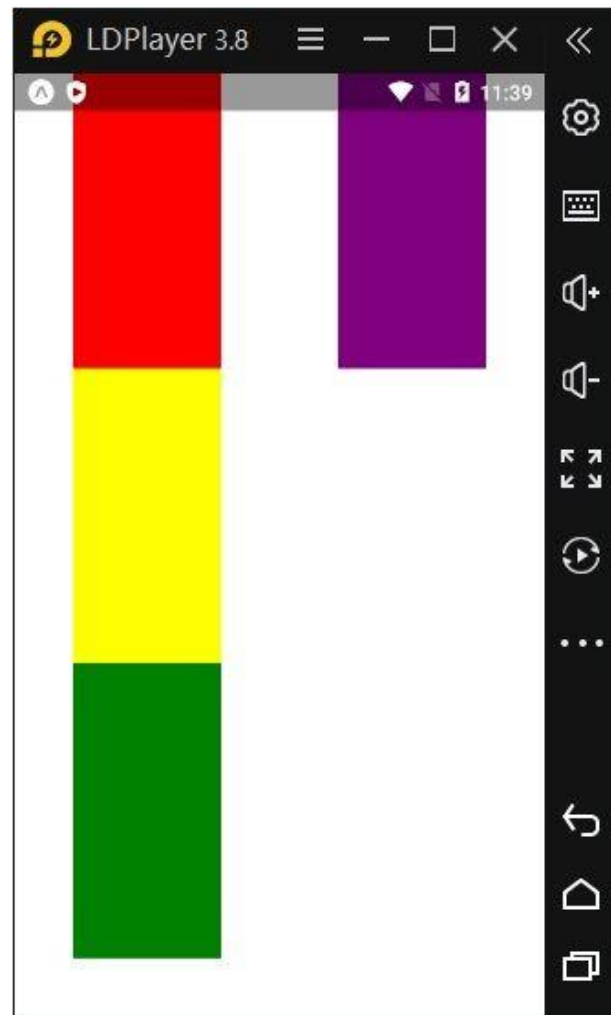
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "wrap",
    alignContent: "flex-end"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 200
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 200
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 200
  },
  roxo: {
    backgroundColor: 'purple',
    width: 100,
    height: 200
  }
});
```



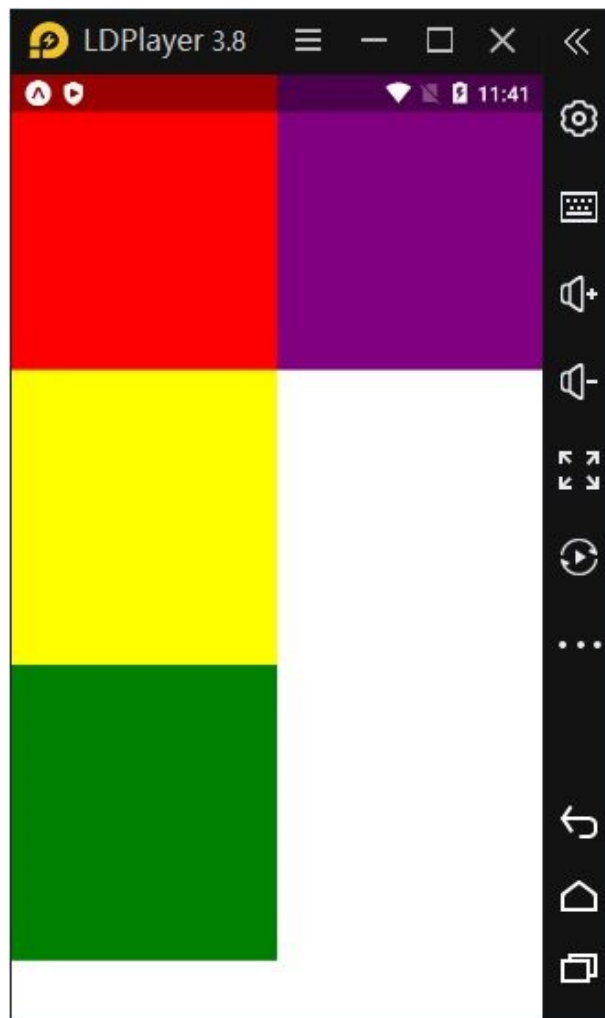
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "wrap",
    alignContent: "space-between"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 200
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 200
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 200
  },
  roxo: {
    backgroundColor: 'purple',
    width: 100,
    height: 200
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "wrap",
    alignContent: "space-around"
  },
  vermelho: {
    backgroundColor: 'red',
    width: 100,
    height: 200
  },
  amarelo: {
    backgroundColor: 'yellow',
    width: 100,
    height: 200
  },
  verde: {
    backgroundColor: 'green',
    width: 100,
    height: 200
  },
  roxo: {
    backgroundColor: 'purple',
    width: 100,
    height: 200
  }
});
```



```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "wrap",
    alignContent: "stretch"
  },
  vermelho: {
    backgroundColor: 'red',
    height: 200
  },
  amarelo: {
    backgroundColor: 'yellow',
    height: 200
  },
  verde: {
    backgroundColor: 'green',
    height: 200
  },
  roxo: {
    backgroundColor: 'purple',
    height: 200
  }
});
```

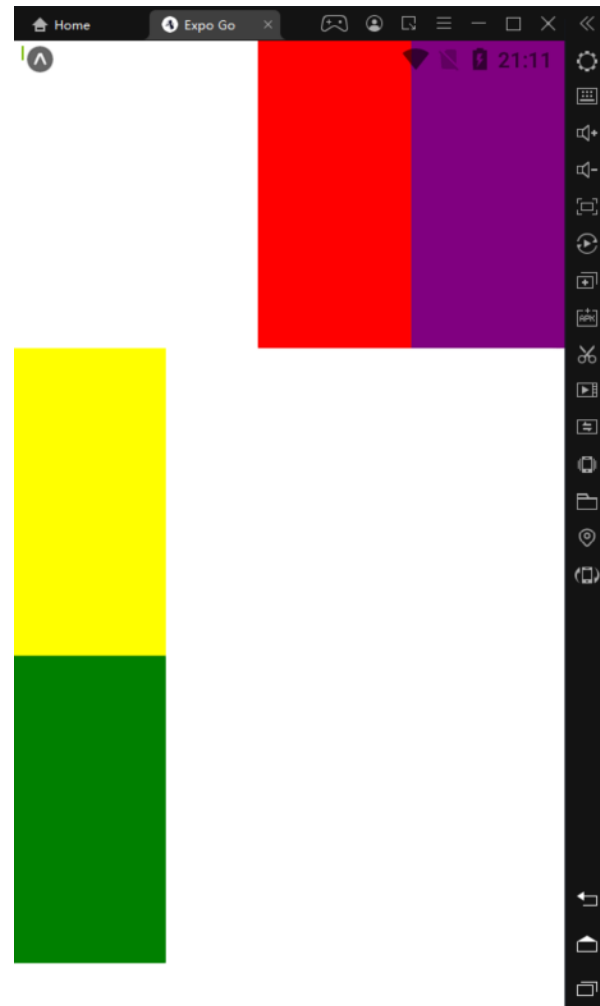


Align Self

Tem as mesmas opções e efeito que **alignItems**, mas em vez de afetar os filhos dentro de um contêiner, **ele aplica esta propriedade a um único filho para alterar seu alinhamento em seu pai**

alignSelf substitui qualquer opção definida pelo pai com **alignItems**

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: "column",
    flexWrap: "wrap",
    alignContent: "space-between",
  },
  vermelho: {
    backgroundColor: "red",
    alignSelf: "flex-end",
    width: 100,
    height: 200,
  },
  amarelo: {
    backgroundColor: "yellow",
    width: 100,
    height: 200,
  },
  verde: {
    backgroundColor: "green",
    width: 100,
    height: 200,
  },
  roxo: {
    backgroundColor: "purple",
    width: 100,
    height: 200,
  },
});
```



Mais informações

Para maiores informações você pode acessar a documentação oficial do React Native:

<https://reactnative.dev/docs/flexbox>

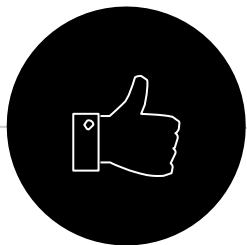
Game Play do Flexbox

<https://flexboxfroggy.com/>

Mais informações

Framework para layout estilo bootstrap para o React Native:

<https://gluestack.io/>



Obrigado!

Dúvidas ?

Você pode me encontrar em:

- jackson.meires@ifsc.edu.br



@jacksonm